

Arquitectura de Influencia en Ecosistemas de Consumo

Sergio Escalante Presa

2026-06-06

Contenido

Introducción	1
1 Bloque I: Análisis de Componentes Principales (PCA).	2
1.1 Carga del dataset.	2
1.2 Observación y preparación del dataset.	3
1.2.1 Métricas incluidas.	4
1.2.2 Posibles métricas interesantes:	4
1.3 ¿Tiene sentido aplicar PCA a nuestra tabla?.	5
1.4 Aplicación de PCA para reducir la dimensionalidad de los datos.	6
1.4.1 Scree Plot: ¿Cuánta información aporta cada componente principal? .	8
1.4.2 El Biplot del dataset.	9
1.5 Conclusión: ¿Qué significa cada componente?	10
2 Bloque II: Análisis de Redes Sociales (SNA).	11
3 Validación Cruzada de Métodos: Comparación de resultados obtenidos en PCA y SNA.	12
4 Bloque III: Clasificación Basada en Enlaces (Link-based).	13
5 Anexo: Diario de Interacción con IA.	14
5.1 Prompt 1: Configuración libro cuarto.	14
5.1.1 Motivo.	14
5.1.2 Prompt (Modelo Opus 4.8 Extra).	14
5.1.3 Output.	14
5.1.4 Valoración.	16
5.2 Prompt 2: Generación archivo theme.scss.	16
5.2.1 Motivo.	16
5.2.2 Prompt (Modelo Opus 4.8 Extra).	17
5.2.3 Output.	17
5.2.4 Valoración.	17

Introducción

En este proyecto de **Aprendizaje Basado en Proyectos (ABP)** se pretende que **actuéis** como analistas de datos para entender cómo se estructuran las relaciones entre productos y usuarios en Amazon.

El objetivo es que **apliquéis** las técnicas vistas en clase, en particular, de reducción de datos, grafos y clasificación para predecir el comportamiento del mercado.

1 Bloque I: Análisis de Componentes Principales (PCA).

En esta primera fase, el objetivo es extraer las dimensiones latentes que definen el éxito de un producto.

- **Tarea:** Reducción de dimensionalidad. Construir una tabla de métricas por producto (Rating promedio, número de reseñas, precio, longitud promedio del texto de la reseña).
- **Análisis:** Aplicar PCA para reducir estas variables a 2 o 3 componentes principales.
- **Interpretación:** ¿Qué representan los ejes? (Ej: ¿El PC1 separa productos “Premium” de “Ventas Masivas”?). Debéis justificar la retención de varianza con un Scree Plot. Si solo hay código no se evalúa, debéis ir extrayendo conocimiento en cada paso, **interpretad**, **analizad** resultados. Justificad la retención de varianza mediante el criterio de Kaiser o el gráfico de sedimentación, e **interpretad** qué significan físicamente los nuevos ejes.

1.1 Carga del dataset.

Primero, cargaremos el dataset `Reviews.csv` con la librería `data.table` ya que es más rápida y eficiente que los métodos y librerías convencionales.

```
library(data.table)

# Usaremos esta semilla para todo el proyecto
set.seed(2004)

reviews <- fread(file="./Reviews.csv")
```

1.2 Observación y preparación del dataset.

Una vez tenemos el dataset cargado, vamos a estudiarlo un poco y a generar una tabla con métricas de cada producto. En particular, vamos a observar qué datos hay **incluidos** en el dataset y nos vamos a quedar con las últimas **3000 reseñas**.

Para crear la tabla usaremos **tidyverse**.

```
# Vemos el nombre de las columnas del dataset
colnames(reviews)
```

```
[1] "Id"                "ProductId"         "UserId"
[4] "ProfileName"       "HelpfulnessNumerator" "HelpfulnessDenominator"
[7] "Score"             "Time"              "Summary"
[10] "Text"
```

```
# Creamos la tabla de productos.
library(tidyverse)

productos <- reviews |>
  mutate(
    size_texto=nchar(Text),
    size_summary=nchar(Summary)
  ) |>
  group_by(ProductId) |>
  summarise(numero_reviews=n(), #Obtenemos el no. de reseñas
            rating_promedio=mean(Score), #Valoración media
            size_summary_promedio=mean(size_summary),
            size_texto_promedio=mean(size_texto),
            numero_usuarios=length(unique(UserId)),
            comprado_por_ultima_vez=max(Time) #Para filtrado
  ) |>
  as_tibble()

# Vemos cuántos productos hay.
cat("Número de productos distintos: ", nrow(productos))
```

Número de productos distintos: 74258

Hay demasiados productos, así los filtraremos para quedarnos con los últimos **4000 productos** que han sido reseñados. Para ello usaremos la variable `comprado_por_ultima_vez`, que después eliminaremos una vez tengamos los productos.

```
productos <- productos |>
  arrange(desc(comprado_por_ultima_vez)) |>
  head(4000) |>
  select(-comprado_por_ultima_vez)
```

Ahora ya tenemos un dataset para aplicarle **PCA**. Antes explicaremos qué métricas hemos incluido en la tabla y se indicará un conjunto de métricas que no se han añadido pero que podrían ser interesantes de estudiar en futuros trabajos.

1.2.1 Métricas incluidas.

- `numero_reviews`: Número de reseñas que tiene un producto.
- `rating_promedio`: Valoración media (sobre 5) que tiene un producto determinado.
- `size_summary_promedio`: Promedio de la longitud (en caracteres) del **resumen** de la reseña.
- `size_texto_promedio`: Promedio de la longitud (en caracteres) del **cuerpo** de la reseña.
- `numero_usuarios`: Número de usuarios que han comprado ese producto.

1.2.2 Posibles métricas interesantes:

1. Una métrica que tenga el promedio (o varianza) del **sentimiento de las opiniones** de los usuarios.
2. Una métrica que incluya el **precio** del producto (no está en el dataset pero podría ser muy interesante de incluir).
3. Una métrica que indique el **tipo** de producto (Informática, Moda, Libros, etc.). Tampoco está en el dataset pero podría ser también muy interesante de incluir.
4. Una variable que indique si tiene **envío PRIME** o no (llega en 1 día).
5. Una columna que represente al vendedor del producto (**Amazon** o un **tercero**).

1.3 ¿Tiene sentido aplicar PCA a nuestra tabla?

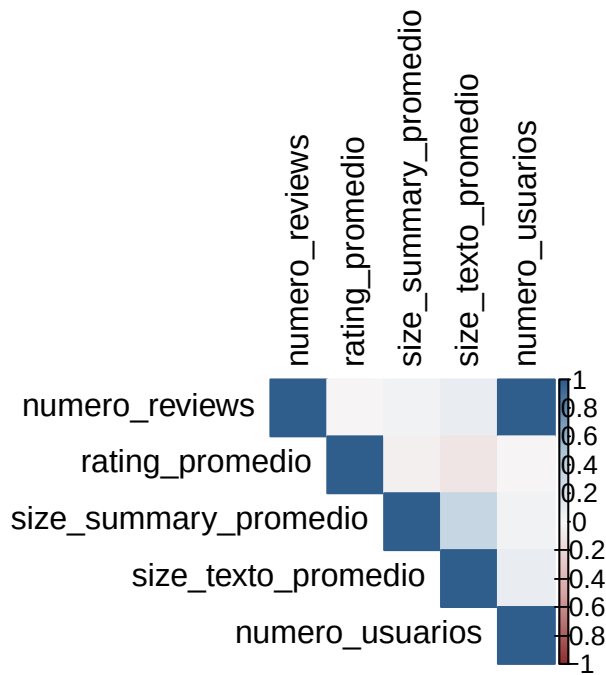
Antes de aplicar PCA, veremos si nuestra tabla tiene **variables correlacionadas**. En caso de que no las tuviera, **no tendría sentido** aplicar PCA ya que no simplificaría los datos más. Para esto usaremos la librería `corrplot`.

Para calcular la correlación y, si tiene sentido, aplicar PCA, **eliminaremos** la columna `ProductId`, ya que solo puede haber valores numéricos en la variable.

```
productos_limpio <- productos |>
  select(-ProductId)

library(corrplot)

corrplot(
  cor(productos_limpio),
  method = "color",
  type = "upper",
  col = colorRampPalette(c("#8C2D2D",
                           "#D9A5A5",
                           "#F7F7F7",
                           "#AFC6D9",
                           "#2F5D8C"))(200),
  tl.col = "black"
)
```



Podemos observar que hay pocos pares de variables muy correlacionados. Sin embargo, hay **bastantes pares de variables que tienen una correlación intermedia** (≈ 0.4).

Por lo tanto, se concluye en que **merece la pena** aplicar PCA para simplificar los datos.

1.4 Aplicación de PCA para reducir la dimensionalidad de los datos.

Usaremos la función `prcomp()` de la librería `stats`. También estableceremos los hiperparámetros `scale` y `center` a `TRUE` para estandarizar los datos, ya que PCA es **muy sensible** a la unidades de medida.

```
pca_productos <- prcomp(productos_limpio, scale=TRUE, center=TRUE)
```

Y usamos la función `summary()` de la librería `base`, para ver la proporción de la varianza explicada por **cada componente principal**, y **cuánto aporta** cada variable de nuestra tabla principal a la creación de estas nuevas variables.

```
resumen <- summary(pca_productos)

cat("Varianza explicada por cada componente principal:\n")
```

Varianza explicada por cada componente principal:

1 Bloque I: Análisis de Componentes Principales (PCA).

```
resumen$importance
```

	PC1	PC2	PC3	PC4	PC5
Standard deviation	1.424894	1.154576	0.9870556	0.813456	0.02532778
Proportion of Variance	0.406060	0.266610	0.1948600	0.132340	0.00013000
Cumulative Proportion	0.406060	0.672670	0.8675300	0.999870	1.00000000

```
cat("\n\nCuánto aporta cada variable inicial a cada una de las PCs:\n")
```

Cuánto aporta cada variable inicial a cada una de las PCs:

```
resumen$rotation
```

	PC1	PC2	PC3	PC4
numero_reviews	-0.69179510	0.1437317	-0.01039419	0.02336735
rating_promedio	0.03991304	0.2809785	0.95170801	-0.11708912
size_summary_promedio	-0.11881802	-0.6614162	0.28438114	0.68376770
size_texto_promedio	-0.16573312	-0.6646725	0.11467269	-0.71943776
numero_usuarios	-0.69154992	0.1453666	-0.01101649	0.02480217

	PC5
numero_reviews	0.7071822006
rating_promedio	-0.0002058369
size_summary_promedio	-0.0002166258
size_texto_promedio	-0.0015774973
numero_usuarios	-0.7070295307

De aquí podemos sacar varias conclusiones importantes, que después se **reforzarán** con pruebas gráficas:

1. Podemos ver que reduciendo los datos a 2 componentes principales explicaríamos aproximadamente el 67% de la varianza de los datos, pero que **usando las 3 primeras** llegaríamos a explicar **casi el 87%** de los datos. Por tanto, sería recomendable usar las 3 primeras componentes principales en vez de las 2 primeras.
2. También, en esa misma tabla, podemos apreciar que la última componente es prácticamente **inútil**, ya que solo explica un **0.01% de la varianza**, aproximadamente.

1 Bloque I: Análisis de Componentes Principales (PCA).

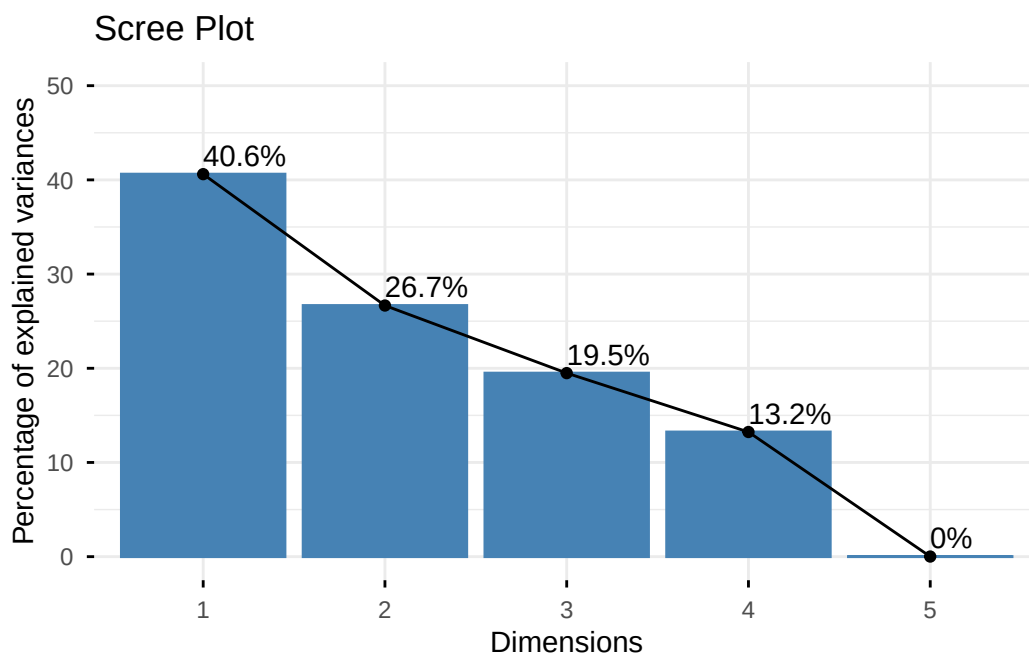
3. Se puede observar de la segunda tabla que, la primera componente tiene una fuerte influencia de las variables `numero_reviews` y `numero_usuarios`; variables que estaban **muy correlacionadas previamente**.
4. La segunda componente está formada principalmente por las variables `size_summary_promedio` y `size_texto_promedio`.
5. La tercera está creada casi exclusivamente por el `rating_promedio`.
6. La cuarta y quinta componente principal tiene una distribución de la aportación de cada variable a su creación **similar a la primera y segunda**. La diferencia es que ahora, esas variables más importantes **dapuntan a un sentido diferente**, es decir, una de ellas tiene coeficiente negativo, mientras que la otra tiene coeficiente positivo.

1.4.1 Scree Plot: ¿Cuánta información aporta cada componente principal?

Representaremos el ScreePlot con la información obtenida en la primera mostrada antes. Para ello usaremos la función `fviz_eig()` de la librería `factoextra`.

```
# Creamos el scree plot
library(factoextra)

fviz_eig(pca_productos,
         addlabels = TRUE,
         ylim = c(0, 50),
         title = "Scree Plot")
```

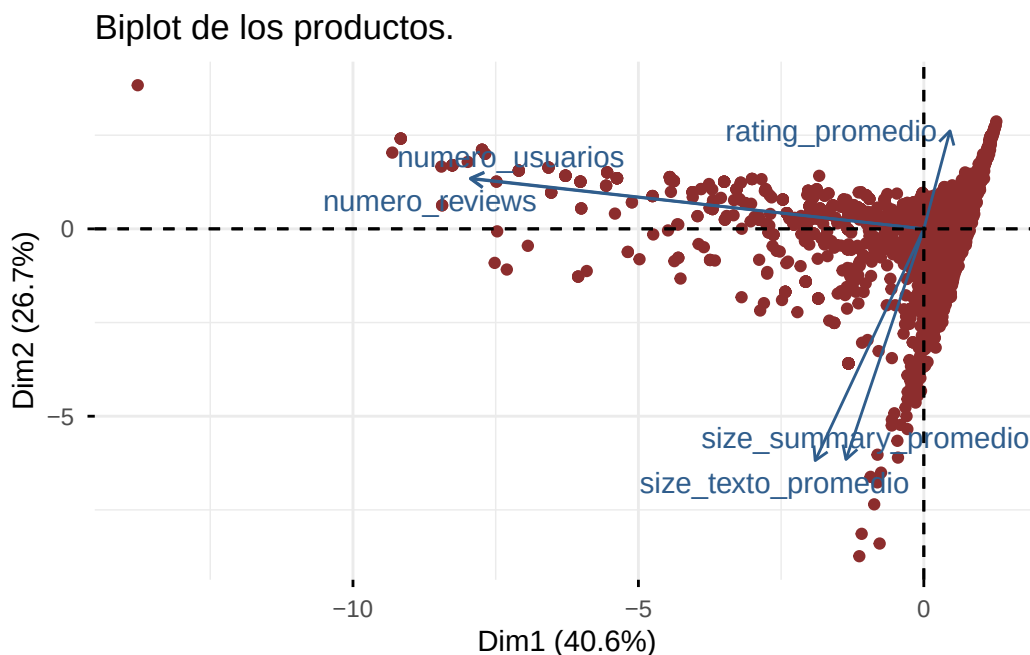


Podemos observar que **no hay un codo muy marcado**. Aún así, sería recomendable coger **mínimo 2 componentes, y como máximo 3**; aunque cogiendo 4 componentes tendríamos casi el 100% de la varianza explicada. Una conclusión evidente es que la quinta componente es **insignificante**.

1.4.2 El Biplot del dataset.

Ahora observaremos el Biplot del dataset con los datos de la segunda tabla mostrada. Para ello usaremos la función `fviz_pca_biplot()` de la librería `factoextra` (cargada previamente).

```
fviz_pca_biplot(pca_productos,  
  col.var = "#2F5D8C",  
  col.ind = "#8C2D2D",  
  repel=TRUE,  
  geom="point",  
  title = "Biplot de los productos.")
```



De esta gráfica podemos sacar 2 cosas en claro muy interesantes.

La primera es que **cuantas más reseñas tenga el producto, mejor**. Esto se puede deber a que un usuario puede preferir un producto con muchas reseñas a uno con pocas, ya que la primera opción es **más segura**.

La segunda es que **la valoración media importa**, pero si sus reseñas son muy largas

contrarresta esa buena valoración. Esto tiene un motivo, las personas que se fijan en si un producto es bueno (alta valoración), la mayoría también quieren leer las opiniones de otros usuarios. Sin embargo, si estas reseñas son muy largas, el usuario se puede ver **sin ganas de leerlas** y decidirse por **otras opciones** con valoraciones similares pero con **reseñas más directas**.

1.5 Conclusión: ¿Qué significa cada componente?

Aquí se dará una posible interpretación de qué significa cada componente:

La primera componente es clara. Representa el número de reseñas que tiene el producto.

La segunda componente es más interesante, ya que representa **cómo de bien reseñado está un producto**. Para esta componente no solo importa la valoración media que tiene el producto, sino que también importa cómo de fácil es leer las opiniones de otros usuarios, penalizando reseñas excesivamente largas. Sería interesante ver cómo se relacionaría el sentimiento promedio de las reseñas con esta variable.

La tercera componente, que no se ha graficado, también tiene una interpretación clara (sacada de la segunda tabla). Esta componente representa simplemente la valoración promedio del producto.

FIN DEL BLOQUE 1

2 Bloque II: Análisis de Redes Sociales (SNA).

3 Validación Cruzada de Métodos:

**Comparación de resultados obtenidos en
PCA y SNA.**

4 Bloque III: Clasificación Basada en Enlaces (Link-based).

5 Anexo: Diario de Interacción con IA.

5.1 Prompt 1: Configuración libro quarto.

5.1.1 Motivo.

Se le ha pedido que genere el apartado `format`: del libro quarto con el que se va a desarrollar el proyecto.

5.1.2 Prompt (Modelo Opus 4.8 Extra).

```
créame un muy buen campo apartado de format para un trabajo científico
y profesional. El documento es un libro de quarto, solo necesito el
campo de format y debe compilar en pdf y html. Tiene que producir
una salida PROFESIONAL, ELEGANTE y MINIMALISTA
```

5.1.3 Output.

Aquí tienes un bloque format pensado para un Quarto book con doble salida (PDF + HTML), buscando ese aire sobrio y tipográficamente cuidado:

```
html:
  theme: cosmo      # limpio y neutro; alternativas: litera, flatly
  fontsize: 1.05em
  linestretch: 1.6      # interlineado generoso = más legible
  highlight-style: github # resaltado de código discreto
  code-copy: true
  code-overflow: wrap
  code-block-bg: true
  code-block-border-left: "#d0d0d0"
  toc: true
```



```
toc-depth: 3
toc-title: "Contenido"
number-sections: true
anchor-sections: true
smooth-scroll: true
link-external-icon: true
link-external-newwindow: true
fig-align: center
fig-cap-location: bottom
tbl-cap-location: top
df-print: paged           # tablas de datos paginadas y limpias
grid:                     # proporciones equilibradas de página
  sidebar-width: 260px
  body-width: 820px
  margin-width: 260px

pdf:
  documentclass: scrbook # KOMA-Script: el estándar para libros serios
  classoption: [oneside] # usa [twoside] si lo vas a encuadernar
  papersize: a4
  geometry:
    - top=25mm
    - bottom=25mm
    - inner=30mm
    - outer=25mm
    - heightrounded
  fontsize: 11pt
  linestretch: 1.4
  pdf-engine: xelatex # tipografía moderna y soporte de fuentes
  colorlinks: true
  linkcolor: black      # enlaces internos discretos
  urlcolor: "RoyalBlue"
  citecolor: "RoyalBlue"
  toc: true
  toc-depth: 3
  toc-title: "Contenido"
```

```
number-sections: true
highlight-style: github
code-block-bg: "#f7f7f7"
fig-cap-location: bottom
tbl-cap-location: top
keep-tex: false
# mainfont: "TeX Gyre Pagella" # descoméntalas para look tipo Palatino
# sansfont: "TeX Gyre Heros"
# monofont: "JetBrains Mono"
```

Tres cosas que conviene que sepas:

El PDF necesita una distribución LaTeX. Si no la tienes, instala TinyTeX con `quarto install tinytex` (es ligera y suficiente). Con xelatex y las fuentes comentadas, por defecto usa Latin Modern, que ya viene con LaTeX y compila sin instalar nada extra. Las fuentes están comentadas a propósito. Si las activas, deben estar instaladas en tu sistema o falla la compilación. TeX Gyre Pagella/Heros suelen venir con TeX Live; JetBrains Mono la tendrías que instalar aparte. Esa terna (serif elegante + sans humanista + mono legible) es la que más eleva el resultado, pero pruébalo solo cuando tengas las fuentes. `oneside` vs `twoside`. Lo dejé en `oneside` porque para lectura en pantalla evita los márgenes asimétricos y las páginas en blanco entre capítulos. Si el destino es imprenta y encuadernado, cambia a `twoside` y el `geometry` con `inner/outer` ya te dará márgenes correctos para el lomo.

Si quieres dar un paso más en la parte HTML, puedo prepararte un `theme.scss` con tu paleta y tipografía para afinar colores, espaciados y bloques de código por encima de lo que permite el YAML.

5.1.4 Valoración.

La parte del `format` me pareció correcta, la probé y me gustó el resultado. Sin embargo la conversación siguió para que me generara ese archivo `theme.scss` que me propuso.

5.2 Prompt 2: Generación archivo `theme.scss`.

5.2.1 Motivo.

Se le ha pedido que genere el archivo `theme.scss` del libro quarto con el que se va a usar para generar el html del proyecto. Este archivo lo propuso la IA en la primera interacción.

5.2.2 Prompt (Modelo Opus 4.8 Extra).

```
vale, creame un scss para la parte de html que sea igual de elegante,  
minimalista y profesional
```

5.2.3 Output.

Para activarlo, en tu `_quarto.yml` cambia la línea del tema dentro de `format: html::`

```
theme: [cosmo, theme.scss]
```

El orden importa: `cosmo` aporta la base de Bootstrap y tu `theme.scss` la sobrescribe encima. Guarda el archivo en la raíz del proyecto (junto al `_quarto.yml`).

Tres decisiones de diseño que conviene que entiendas:

El sistema tipográfico IBM Plex (sans + serif + mono) es lo que más eleva el resultado. El cuerpo va en sans para leer cómodo en pantalla, pero los títulos van en serif: ese contraste serif/sans es el truco editorial que da el aire de “libro científico” sin recargar nada. Al ser una sola familia coordinada, todo se ve coherente. Se cargan desde Google Fonts, así que la primera compilación necesita conexión.

Un solo acento (azul tinta `#15506b`). En minimalismo, la elegancia viene de la restricción: fondo blanco, texto casi negro y un único color que marca enlaces, TOC activo y citas. Si quieres un tono más académico clásico, cambia `--accent` y `$link-color/$primary` a un granate (`#7a1f2b`). Las tablas usan estilo “booktabs” (solo líneas horizontales gruesas arriba/abajo y bajo la cabecera, sin rejilla vertical). Es la convención de las publicaciones científicas serias y es lo que hace que una tabla se vea profesional en lugar de “de hoja de cálculo”.

Si más adelante quieres que el PDF y el HTML compartan exactamente la misma paleta de acento, puedo darte el `include-in-header` de LaTeX equivalente para que los enlaces del PDF usen ese mismo azul tinta.

[Más fichero `theme.scss` usado en el proyecto]

5.2.4 Valoración.

De nuevo, me pareció correcto el resultado que produjo, así que no se siguió preguntando.